# Version control and collaboration with Git

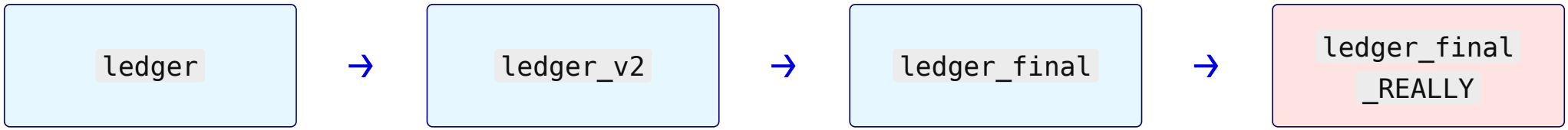
## Software Management & Development




IE University · MFT & MCSDI

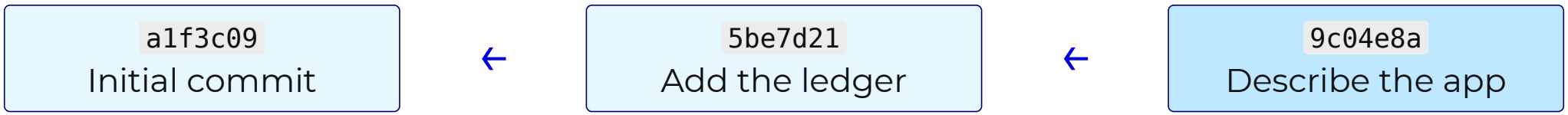
# Version control

# Without version control




- Which copy works? What changed between two of them, and why?
- Four people edit the same files: who overwrote whose work?

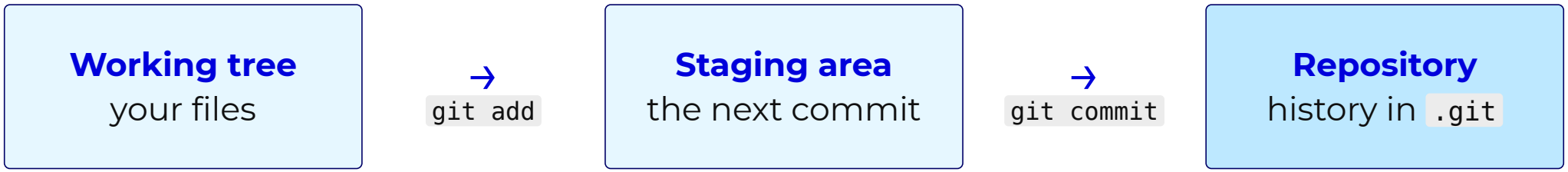
# Commit




A snapshot of every tracked file, with author, date, message and a parent.

> The hash is computed from the contents and the parent: an old commit can't change without everyone noticing.

# The three areas




`git status` shows where each change is. `git diff --staged` shows what you're about to record.

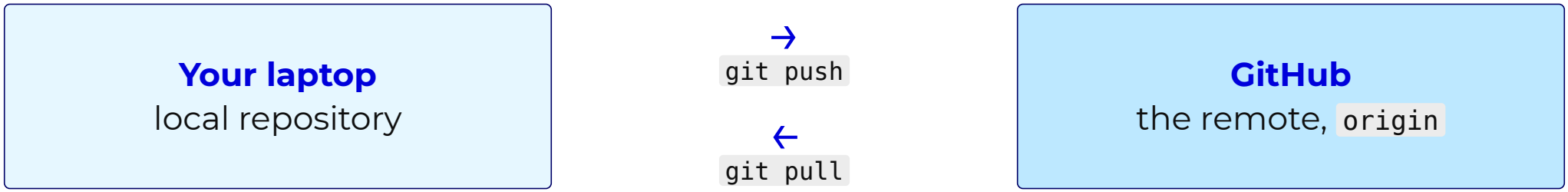
# What goes in the repository

| **Commit** | **Keep out (`.gitignore`)** |
| --- | --- |
| `pyproject.toml`, `uv.lock`, `.python-version` | `.venv/`: rebuilt by `uv` |
| `src/`, `README.md` | `__pycache__/`, `.ruff_cache/` |
| sample data the code needs | `.env` with passwords and keys |

> A secret that was committed stays in the history. Revoke it and create a new one.

# GitHub

# Local and remote




- `git clone <url>`: copy a repository with its whole history
- Your commits reach GitHub only when you push; theirs reach you only when you pull

> `git pull` = `git fetch` (download) + `git merge` (combine).

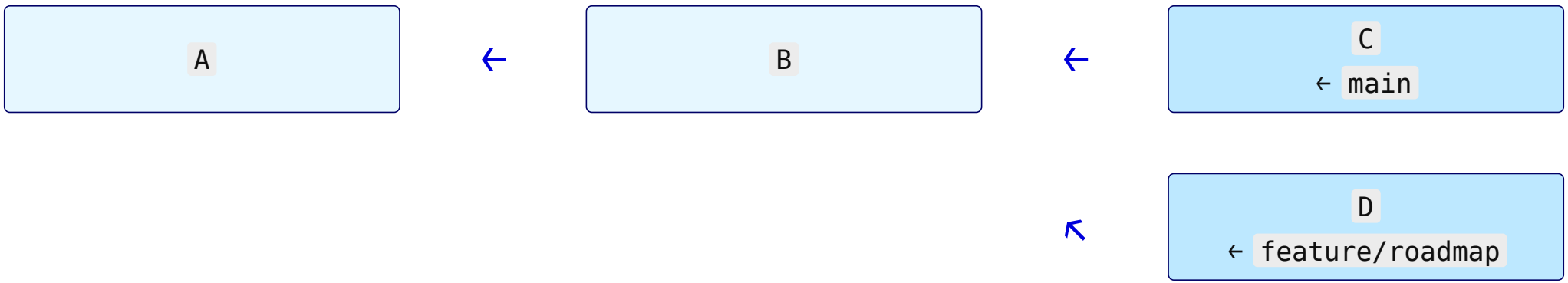
# Signing in to GitHub

```
gh auth login        # GitHub.com → HTTPS → Yes → web browser
gh auth status
```

- The GitHub CLI signs you in once, through the browser
- Git uses that login for every push and pull: no passwords, no keys

# Branches

# Branch




A branch is a name that points to a commit, and moves forward when you commit on it.

> `git switch -c feature/roadmap` creates a branch and moves to it. `main` holds the official history.

# Merge

**Fast-forward**
only one side has new commits:
`main` just moves forward

**Merge commit**
both sides have new commits:
a commit with two parents

Git combines changes on its own when they touch different lines.

# Conflict

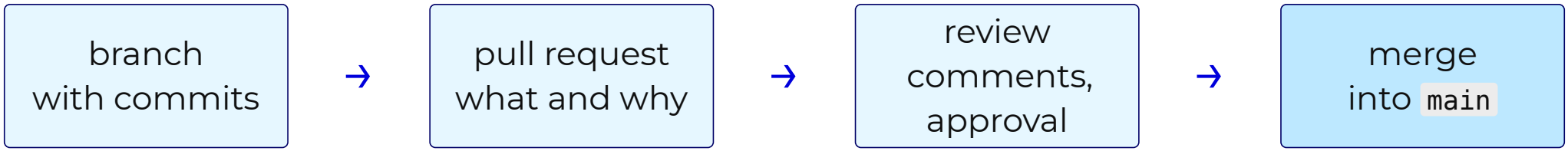
Both sides changed **the same line**. Git stops and shows both versions:

```
<<<<<<< HEAD
Personal finance ledger: balances, spending and monthly reports.
=======
Personal finance ledger for the command line.
>>>>>>> c3b83bd2a4f5...
```

- Edit the file so it says what it should, and delete the markers
- `git add` the file, then `git commit` to finish the merge

> A conflict is not an error: Git refuses to guess what the text means.

# Pull request




At least two people have read every change before it reaches `main`.

> We merge with **Create a merge commit**: the history shows each branch and when it was merged.